6.6 Subclassing API (Lập trình Hướng đối tượng)



Mô hình Tĩnh: Sequential & Functional API	Mô hình Động: Subclassing API
• Dạng Tuyên bố (Declarative). Đồ thị cố định.	• Cấu trúc linh hoạt. Hỗ trợ vòng lặp, if/else trực tiếp trong kiến trúc.

```
class WideAndDeepModel(keras.models.Model):
    def __init__(self, units=30, activation="relu", **kwargs):
        super().__init__(**kwargs)
        self.hidden1 = keras.layers.Dense(units, activation=activation)
        self.hidden2 = keras.layers.Dense(units, activation=kwargs)
        self.main_output = keras.layers.Dense(1)

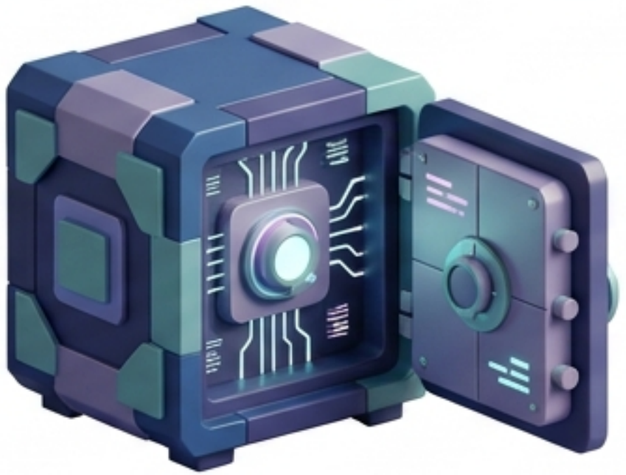
    def call(self, inputs):
        # Dinh nghia cach truyen tien tai day
        hidden1 = self.hidden1(inputs)
        hidden2 = self.hidden2(hidden1)
        concat = keras.layers.concatenate([inputs, hidden2])
        return self.main_output(concat)
```

Gemini Notebook

7.1 Lưu và Khôi phục Mô hình

Quá trình huấn luyện Mạng Nơ-ron tốn rất nhiều thời gian (Có thể vài ngày). Bạn cần lưu lại!

Định dạng HDF5 (.h5)



- Toàn bộ Trọng số
- Cấu trúc
- Optimizer

Lưu Mô Hình

```
# Luu model
model.save("my_keras_model.h5")
```

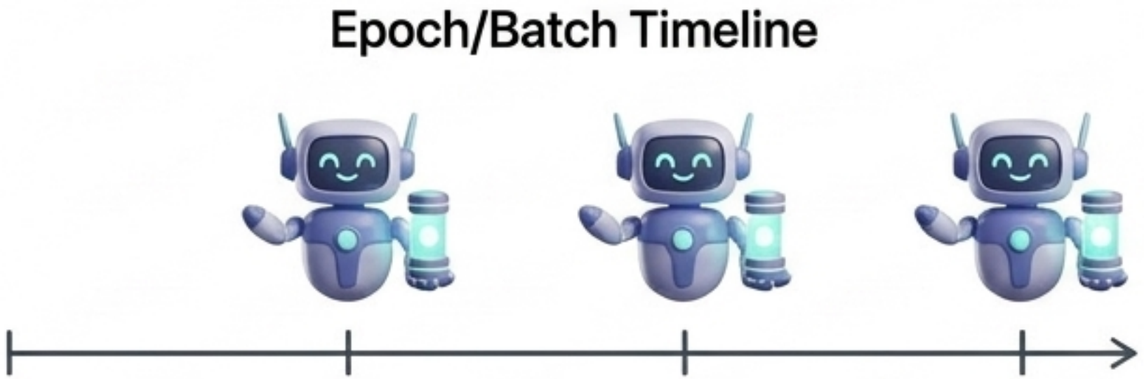
Khôi Phục Mô Hình

```
# Phuc hoi model
model = keras.models.load_model("my_keras_model.h5")
```

Gemini Notebook

7.2 Callbacks: Trợ lý huấn luyện

Keras cho phép gọi các hàm **Calllback ở mỗi cuối Epoch** hoặc cuối **Batch**.



ModelCheckpoint

- Tự động lưu mô hình mỗi khi nó Đạt điểm cao nhất trên tập Validation.
- **Mục đích:** Tránh mất mô hình tốt nhất nếu bị sập nguồn.

```
checkpoint_cb = keras.callbacks.ModelCheckpoint(
    "my_best_model.h5",
    save_best_only=True
)

history = model.fit(
    X_train, y_train,
    epochs=10,
    validation_data=(X_valid, y_valid),
    callbacks=[checkpoint_cb]
)
```

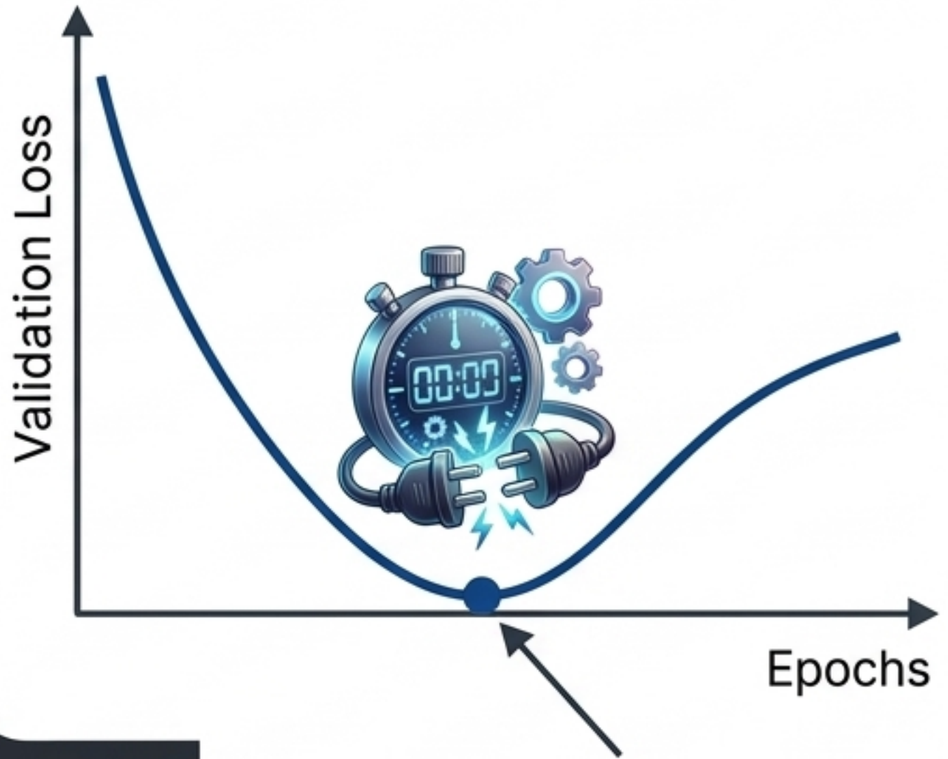
Gemini Notebook

7.3 Dừng sớm (Early Stopping)

Không ai biết cần đặt epochs bao nhiêu là vừa. Đặt lớn quá sẽ gây Overfitting.

```
early_stopping_cb = keras.callbacks.EarlyStopping(
    patience=10,
    restore_best_weights=True
)
# patience=10 nghia la doi them 10 epoch xem co hy vong khong.

history = model.fit(
    X_train, y_train,
    epochs=1000, # Dat epoch 1000 cho thoai mai
    validation_data=(X_valid, y_valid),
    callbacks=[checkpoint_cb, early_stopping_cb]
)
```



Ear'lyStopping: Tự động Rút phích cắm dừng huấn luyện nếu nhận thấy Loss trên tập Validation không còn cải thiện.

Gemini Notebook

7.4 TensorBoard: Công cụ Giám sát tối thượng

Giao diện đồ họa siêu cấp của TensorFlow để theo dõi quá trình huấn luyện trực tiếp.

Real-time



Vẽ đồ thị Loss, Accuracy theo thời gian thực.

Debugging



Cực kỳ hữu ích để gỡ lỗi các mạng khổng lồ.



Architecture



Hiển thị sơ đồ đồ thị kết nối Lớp (Graph).

Terminal Box

Ghi log bằng cách truyền vào callbacks:

```
keras.callbacks.TensorBoard(log_dir="logs")
```

Gemini Notebook

8.1 Tinh chỉnh Siêu Tham số (Hyperparameters)

Mạng Nơ-ron có độ linh hoạt tuyệt đối -> Hàng tá thông số cần tinh chỉnh! (Ví dụ: 2 lớp hay 100 lớp ẩn?).



Chiến lược Tìm kiếm
Phương pháp Cơ bản: GridSearchCV (Duyệt toàn bộ).
Mạng Sâu (Khuyên dùng): RandomizedSearchCV - Hiệu quả hơn nhiều cho Deep Learning.

Tối ưu hóa Chuyên sâu

Nâng cấp kỹ thuật với công cụ chuyên biệt Keras Tuner.

Gemini Notebook

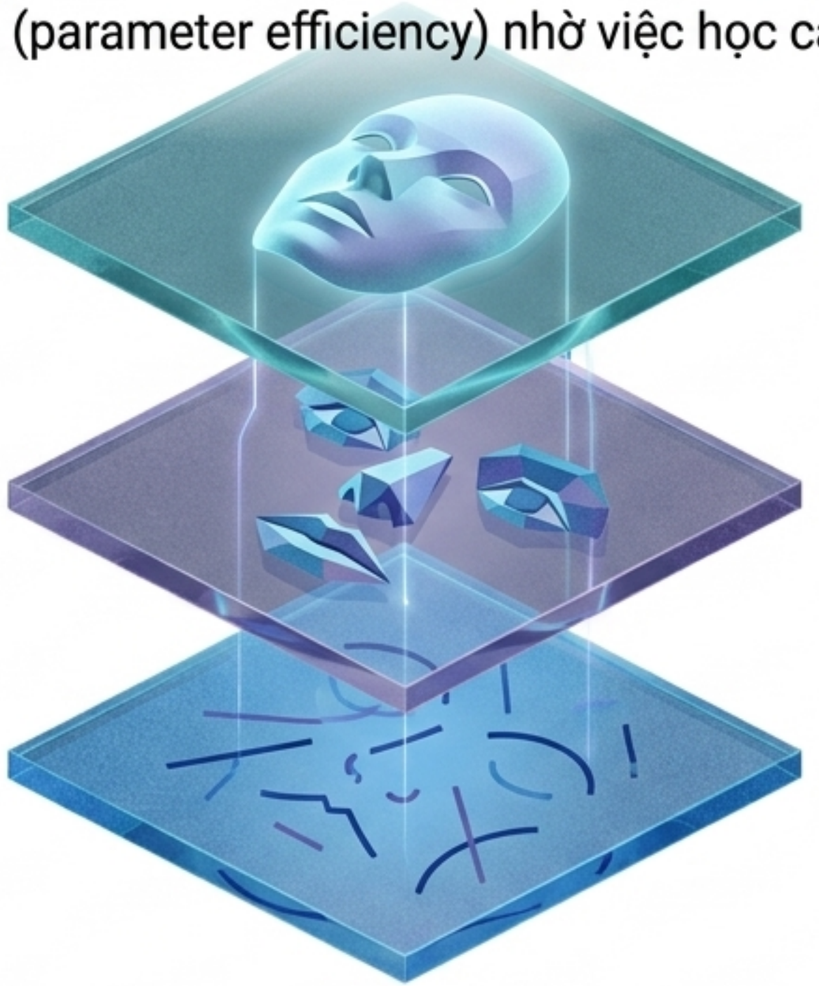
8.2 Số lượng Lớp Ẩn (Hidden Layers)

Mạng 1 lớp ẩn rộng có thể xấp xỉ bất kỳ hàm nào, nhưng rất tốn tham số.
Mạng Sâu có hiệu suất tham số lớn hơn (parameter efficiency) nhờ việc học cách phân cấp dữ liệu.

Lớp 3: Gộp thành khuôn mặt.

Lớp 2: Vẽ thành mắt/mũi/miệng.

Lớp 1: Học vẽ đoạn thẳng.



Visual Hierarchy (Nhận diện mặt)

Kinh nghiệm (Rule of Thumb)

- Bắt đầu với **1 hoặc 2 lớp ẩn** (tốt cho đa số bài toán).
- Ảnh phức tạp: Cần hàng chục lớp.

Gemini Notebook

8.3 Số lượng Nơ-ron trên mỗi Lớp Ẩn

Cấu trúc Cổ điển (Hình phễu)



- **Nơ-ron giảm dần:** 300 -> 100 -> 50.
- **Mục đích:** Tinh lọc dữ liệu.

Cấu trúc Hiện đại (Hình trụ)



- **Đặt số nơ-ron bằng nhau** (VD: tất cả 150).
- **Kết quả:** Hiệu suất tương tự hoặc tốt hơn, giảm bớt cấu hình tham số.

Tactical Tip

Mẹo vặt Chiến thuật

Hãy chọn mô hình có số lớp và số nơ-ron thừa thãi nhiều hơn mức cần thiết, sau đó sử dụng Early Stopping và Dropout để ngăn Overfitting.

Gemini Notebook

8.4 Tốc độ học (Learning Rate)

Tốc độ học (lr) là siêu tham số QUAN TRỌNG NHẤT. Nếu chỉ có thời gian tinh chỉnh 1 tham số, hãy tinh chỉnh cái này!



(Zone: Quá nhỏ)

Mất rất nhiều ngày tháng để huấn luyện.

Giá trị khởi điểm kinh điển: 10^{-3} hoặc 10^{-4}

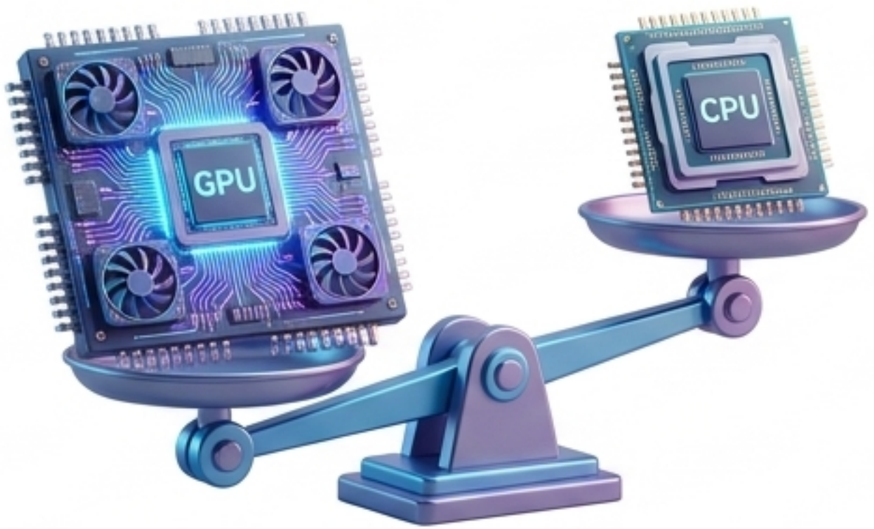
(Zone: Quá lớn)

Mạng nhảy qua lại, không thể hội tụ (Diverge).

Gemini Notebook

8.5 Kích thước Lô (Batch Size) & Optimizer

Kích thước Lô (Batch Size: 32 - 128)



- **Batch Size To:** Huấn luyện nhanh nhờ GPU, nhưng **nguy cơ rơi vào tối ưu cục bộ.**
- **Batch Size Nhỏ:** Tốn CPU, thời gian lâu nhưng mang tính nhảy múa cao, dễ thoát hố sâu cục bộ.

Optimizer (Bộ tối ưu)



- **Thuật toán Adam** (Sẽ học ở chương 11) được chứng minh tốt hơn nhiều so với SGD cổ điển.
- **Thực tiễn:** Đang được dùng mặc định ở hầu hết bài toán Học sâu.

Gemini Notebook



HỎI & ĐÁP

Gemini Notebook